

LEFT JOIN / ON / GROUP BY / COUNT 知识点总结

这份文档整理了我们上面讨论过的 SQL 关联查询知识点，重点围绕 LEFT JOIN ... ON ...、COUNT、GROUP BY 和它们在 GORM 里的对应写法。

面向当前项目 例子来自 dynasties / poems 适合在 DBeaver 验证

整理日期：2026-07-11

1. 一句话先记住

LEFT JOIN：以左表为主，把右表能匹配上的数据补过来；匹配不上也保留左表这一行。

ON：说明左右两张表按照什么条件匹配。

GROUP BY：把结果按某个字段分组，再分别做统计。

COUNT：计算数量，常和 GROUP BY 一起用。

2. 基本语法是什么意思

```
SELECT 列  
FROM 左表  
LEFT JOIN 右表 ON 左表.字段 = 右表.字段
```

可以翻译成一句中文：

先从左表取数据，再到右表里找能对应上的记录，拼在一起；找不到也不删左表记录。

3. 什么叫 LEFT JOIN

因为它是“左连接”，左边的表是主角。

- 左表：必须尽量完整保留。
- 右表：只是补充信息。
- 右表没有匹配项：右表列会变成 NULL，但左表记录还在。

4. 用朝代和诗举例

4.1 原始表

dynasties.id		dynasties.name	
1		唐	
2		宋	
3		元	
poems.id	poems.title		poems.dynasty_id

10	静夜思	1
11	春晓	1
12	念奴娇	2

4.2 执行 LEFT JOIN

```
SELECT *
FROM dynasties
LEFT JOIN poems ON dynasties.id = poems.dynasty_id;
```

dynasties.id	dynasties.name	poems.id	poems.title	poems.dynasty_id
1	唐	10	静夜思	1
1	唐	11	春晓	1
2	宋	12	念奴娇	2
3	元	NULL	NULL	NULL

这里最关键的是最后一行。元 没有匹配到诗，但仍然保留。这就是 LEFT JOIN 和 INNER JOIN 的核心区别。

5. ON 是干嘛的

```
ON dynasties.id = poems.dynasty_id
```

这句的意思是：

把 dynasties 表里的 id 和 poems 表里的 dynasty_id 对起来，数值相同就认为这两行有关联。

可以理解成“匹配规则”或者“关联条件”。没有 ON，数据库就不知道左表和右表应该如何对接。

6. LEFT JOIN 和 INNER JOIN 的区别

写法	效果
LEFT JOIN	左表全保留，右表有就补，没有就 NULL。
INNER JOIN	只有左右两边都匹配上的记录才保留。

如果要统计“所有朝代，包括暂时没有诗的朝代”，就应该用 LEFT JOIN。

7. COUNT 是干嘛的

```
COUNT(poems.id) AS poem_count
```

这句的意思是：

统计 poems.id 的数量，并把这列临时命名为 poem_count。

注意这里的 `AS poem_count` 只是给查询结果里的这列起别名，不是去修改数据库表结构。

7.1 它统计的是哪张表

它统计的是右表 `poems` 这一侧的 `id`，但不是“脱离上下文统计整张 `poems` 表”，而是统计 **和当前左表分组关联上的 `poems.id` 数量**。

7.2 为什么常写 `COUNT(poems.id)` 而不是 `COUNT(*)`

因为 `LEFT JOIN` 时，没有匹配到的右表列会是 `NULL`。`COUNT(poems.id)` 只统计非空值，所以当某个朝代没有诗时，结果自然是 0。

8. GROUP BY 为什么必须有

```
GROUP BY dynasties.id
```

如果你已经做了连接，结果里同一个朝代会重复出现多次。要想得到“每个朝代有几首诗”，就必须先分组，再统计。

所以 `GROUP BY dynasties.id` 的意思是：

按朝代 `id` 把结果分组，然后对每组分别执行 `COUNT`。

8.1 没有 `GROUP BY` 会怎样

```
SELECT dynasties.*, COUNT(poems.id) AS poem_count
FROM dynasties
LEFT JOIN poems ON dynasties.id = poems.dynasty_id;
```

这类写法在很多数据库里会报错，因为你混用了普通列和聚合列，却没有告诉数据库“按什么粒度统计”。

8.2 按主键 `id` 分组为什么没意义

```
GROUP BY poems_zh_hant.id
```

如果这里的 `id` 是主键，通常每行都唯一。那每组里就只有 1 行，最后 `COUNT` 大多数都会是 1，统计价值很低。

8.3 按 `dynasty_id` 分组就合理

```
SELECT poems_zh_hant.dynasty_id, COUNT(poems_zh_hant.id) AS poems_count
FROM poems_zh_hant
GROUP BY poems_zh_hant.dynasty_id
ORDER BY poems_count DESC;
```

这里是在统计“每个朝代有多少首诗”，因为多首诗会共享同一个 `dynasty_id`。

9. ORDER BY poem_count DESC 是干嘛的

```
ORDER BY poem_count DESC
```

按前面临时算出来的 `poem_count` 列降序排列，也就是：

诗数量最多的朝代排在最前面。

10. 和当前 GORM 代码的对应关系

```
err = r.db.Table(dynastyTable).
    Select(dynastyTable + ".*", COUNT(" + poemTable + ".id) as
    poem_count").
    Joins("LEFT JOIN " + poemTable + " ON " + dynastyTable + ".id = " +
    poemTable + ".dynasty_id").
    Group(dynastyTable + ".id").
    Order("poem_count DESC").
    Scan(&dynastyStats).Error
```

这段 GORM 链式调用对应的大致 SQL：

```
SELECT dynasties.*, COUNT(poems.id) AS poem_count
FROM dynasties
```

```
LEFT JOIN poems ON dynasties.id = poems.dynasty_id

GROUP BY dynasties.id

ORDER BY poem_count DESC;
```

逐句翻译

1. FROM dynasties: 以朝代表为主。
2. LEFT JOIN poems ...: 把诗表关联上来，没有诗的朝代也保留。
3. COUNT(poems.id) AS poem_count: 统计每个朝代关联到多少首诗。
4. GROUP BY dynasties.id: 按朝代分组分别统计。
5. ORDER BY poem_count DESC: 按诗数量从高到低排序。
6. Scan(&dynastyStats): 把结果扫描进 Go 变量。

11. 在 DBeaver 里怎么验证

1. 先确认实际表名和字段名。
2. 把最终 SQL 粘进 SQL Editor，而不是贴 GORM 链式代码。
3. 先跑简单语句，例如 SELECT * FROM dynasties LIMIT 5;。
4. 再跑带连接和统计的完整查询。

推荐直接验证的 SQL:

```
SELECT dynasties.id, dynasties.name, COUNT(poems.id) AS poem_count
FROM dynasties
LEFT JOIN poems ON dynasties.id = poems.dynasty_id
GROUP BY dynasties.id, dynasties.name
ORDER BY poem_count DESC;
```

如果你连接的是 PostgreSQL，通常要像上面这样，把非聚合列显式写进 GROUP BY；如果是 MySQL，行为可能更宽松。

12. 常见误区

- **误区 1:** `LEFT JOIN` 像把右表整体 append 到左表末尾。更准确地说，它是
以左表为主，按匹配关系展开新的结果集。
- **误区 2:** `COUNT(poems.id)` 统计的是 poems 全表总数。实际上它常常是
“当前分组中匹配到的 poems.id 数量”。
- **误区 3:** `AS poem_count` 会给数据库真加一列。实际上它只是查询结果里的
临时别名。
- **误区 4:** `GROUP BY` 主键 `id` 一定有意义。若 `id` 唯一，统计结果通常只是很
多个 1。

13. 最短复习版

`LEFT JOIN`: 左表全保留。

`ON`: 左右表如何匹配。

`COUNT(poems.id)`: 统计每组里右表匹配到的诗数量。

`GROUP BY dynasties.id`: 按朝代分别统计。

`ORDER BY poem_count DESC`: 按统计结果从大到小排序。